1. What am I making?

RADIX ProofPath helps a candidate understand job readiness. Five teammates own five parts: JD Analytics -> Resume Parsing -> Profile Builder -> Talent Check -> Skill Matching. You own part 3.

Save one useful profile for both assessments.

Let a candidate combine reviewed CV information with details the CV may miss. Save it so both assessments use the same candidate record.

Why your part matters

A resume is only a starting point. Hackathons, internships, certificates and preferred roles help create a fuller profile. Later edits must persist correctly.

What the user gives you

Manual form entries, an optional reviewed resume from Module 2 and an optional CV upload.

What you return

A saved candidate profile with its ID and revision, editable background, source-linked claims and twelve separate self-ratings that may stay Unknown.

A small example

Asha adds her name and preferred role, saves, then reloads the page. Her details remain. Importing a CV fills a draft; it does not silently replace her saved edits or assign ability levels.

Start with the working code

This is a working starting point, not an instruction to rebuild an app. Clone the WHOLE repository so shared support is available. Work mainly in profile_builder/. Read this one guide, run it, then improve a problem you can demonstrate.

Your folder has this guide as PDF and Markdown, backend/, frontend/, tests/, samples/ and run.ps1. Shared support is stored once at the root. The Markdown contains the same teaching content and full examples as this PDF.

2. What exactly must I build?

Organizer requirements - keep these working

- Collect basic details, skills, hackathons/internships, certifications, preferred roles and a CV upload.
- Allow manual entry; pre-fill from Resume Parsing where available.
- Validate, save and load a profile in the shared JSON format. JSON means named fields in a text record that other modules can read.
- Run without a live parser, and give the same saved profile to Modules 4 and 5. Use ordinary code; this form does not need an LLM.

Already implemented

The form, direct CV upload, reviewed-resume import, fill-empty/replace choice, create/load/update/delete, local persistence and revision checks already work. The passport shows current claims and source links.

Your development checklist

- 1. Run the form, create a manual profile and reload it. Check all required background fields, not just name and skills.
- 2. Load the supplied reviewed resume through the local synthetic workspace. Import, inspect the draft and save deliberately.
- 3. Test fill-empty versus replace: keep a manual education entry, import again, and confirm it only changes when replacement is explicitly selected.
- 4. Check invalid entries, stale saves, source links and both assessment handoffs. Improve only a reproduced issue; add a test for that issue.

Currently missing or unverified

Live Supabase persistence and official sample acceptance remain unverified. The passport does not keep an immutable historical timeline. Do not promise that feature or rebuild the working form.

Extra ProofPath features

The Living Skill Passport already keeps current claims, quotes and profile revision. Evidence confidence remains separate from twelve optional numeric self-ratings. Adding a quote must never raise a rating automatically.

3. How does it work?

1. Start	**2. Fill the draft**	**3. Add details**
4. Keep ratings separate	**5. Save and reload**	**6. Assess**

- 1. Start: Create a blank profile or choose a reviewed resume. Both paths work without Groq.
- 2. Fill the draft: Import fills supported fields. Existing manual fields are preserved unless Replace is selected.
- 3. Add details: Enter projects, internships, hackathons, certificates, preferred roles and CV reference. Inspect skills and their sources.
- 4. Keep ratings separate: Each official category starts as Unknown (JSON null). A user may explicitly choose a self-rating from 1 to 10.
- 5. Save and reload: The server checks fields and source ownership, saves the profile and returns its current version. A conflicting old save is rejected.
- 6. Assess: Modules 4 and 5 load this same saved ID/version. Career GPS makes a temporary copy and cannot save hypothetical claims here.

Words you will see

Backend = Python server code. Frontend = React browser screen. API = an address the screen calls to send/receive data. A contract/schema is the agreed field list, defined in shared/contracts/models.py. An LLM is a language model that interprets text; Groq runs it. Supabase is the cloud database service.

The shared RADIX categories

COD Coding; DSA Data Structures & Algorithms; OOD Object-Oriented Design; APTI Aptitude; COMM Communication; AI Artificial Intelligence; CLOUD Cloud; SQL SQL; SWE Software Engineering; SYSD System Design; NETW Networking; OS Operating Systems.

Each skill has exactly one category, or OTHER. Descriptive skill families are extra UI filters, not new rated categories. Unknown competency is null, never zero. Confidence describes extraction support, not ability. What-if changes never save a real profile.

4. Inputs and outputs

The user and optional Module 2 provide inputs. Modules 4 and 5 receive the saved Profile.

POST /api/profiles

Send the following complete JSON body to create a simple manual profile. Only name is mandatory on creation; omitted fields receive the defaults shown in the complete response. This example deliberately has no imported claims.

Complete input example: profile_builder/samples/input.json

```
{
  "name": "Asha Sample (synthetic)",
  "email": "asha@example.test",
  "education": ["Invented computing course"],
  "preferred_roles": ["Junior Developer"]
}
```

Important fields

- name / email / background: name is required and cannot be blank. Email may be empty but must have a valid shape if supplied. Education and other multi-entry fields are arrays.
- skills / cv_file: Skills follow the shared Skill fields. cv_file is a private document ID, not a public URL or a path to somebody's computer.
- ratings: Exactly twelve category entries are stored; each level is null (Unknown) or a number 1-10 with self_reported origin. OTHER is not a thirteenth rating.
- id / revision: The server returns the profile identity/version. Keep both on update; do not hardcode the example's generated ID into the UI.

GET /api/profiles lists records; GET /api/profiles/{id} loads one. PATCH /api/profiles/{id} needs the full Profile with its current revision; DELETE removes it. POST /api/profiles/import-resume needs resume_id and profile, with replace_existing=false by default. It returns a draft; save that draft explicitly. POST /api/documents uploads a CV without extracting it.

The next page(s) show the COMPLETE captured response from the existing API, including defaults and empty fields. Long lines wrap for reading. Use samples/output.json for exact machine-readable JSON. IDs, receipt IDs, source hashes and times can differ between runs; do not hardcode them. JSON null means unknown/not present; [] means an empty list.

Complete output example

profile_builder/samples/output.json


```
{
  "schema_version": "1.0",
  "id": "aa2ffcb7-b0ca-414b-8eea-ac261ccef3bc",
  "revision": 1,
  "name": "Asha Sample (synthetic)",
  "email": "asha@example.test",
  "education": ["Invented computing course"],
  "skills": [],
  "hackathons": [],
  "internships": [],
  "certifications": [],
  "preferred_roles": ["Junior Developer"],
  "projects": [],
  "experience": [],
  "achievements": [],
  "cv_file": null,
  "ratings": [
    {"category_code": "COD", "level": null, "origin": "self_reported"},
    {"category_code": "DSA", "level": null, "origin": "self_reported"},
    {"category_code": "OOD", "level": null, "origin": "self_reported"},
    {"category_code": "APTI", "level": null, "origin": "self_reported"},
    {"category_code": "COMM", "level": null, "origin": "self_reported"},
    {"category_code": "AI", "level": null, "origin": "self_reported"},
    {"category_code": "CLOUD", "level": null, "origin": "self_reported"},
    {"category_code": "SQL", "level": null, "origin": "self_reported"},
    {"category_code": "SWE", "level": null, "origin": "self_reported"},
    {"category_code": "SYSD", "level": null, "origin": "self_reported"},
    {"category_code": "NETW", "level": null, "origin": "self_reported"},
    {"category_code": "OS", "level": null, "origin": "self_reported"}
  ]
}
```

5. How do I work on the code?

Install and run - Windows PowerShell

Install Python 3.10+, Node.js 24 LTS (includes npm), Git and an editor such as VS Code. Accept the installers' PATH option, reopen PowerShell and clone the full GitHub repository. Open a terminal at the repository root - the folder containing package.json.

```
.\profile_builder\run.ps1 setup
# Terminal 1:
.\profile_builder\run.ps1 api
# A SECOND terminal, same root:
.\profile_builder\run.ps1 web
```

Open http://127.0.0.1:5173/. Server API help is at http://127.0.0.1:8000/docs. setup creates .venv and installs locked Python/npm dependencies plus Chromium for browser tests. Run setup once; no key is needed. Keep api and web running; Ctrl+C stops your own server. Only one module uses 8000/5173 at a time.

Open these files first

- profile_builder/frontend/ProfileBuilderPage.tsx - Open first. The form, import preview, save button and rating controls live here.
- profile_builder/backend/service.py - Combines reviewed fields, preserves manual entries and checks that source documents belong to the user.
- profile_builder/backend/router.py - Creates, loads, updates, imports and deletes records.
- profile_builder/tests/test_final_merge.py - Examples for import conflicts, replacement and unknown ratings.

shared/contracts/models.py defines Profile, Skill and Rating. shared/integrations/store.py saves records; documents.py handles CVs. Read those if needed, but coordinate any shared storage/schema change with the Scrum Master.

Make one useful change

Read a failing example, locate the responsible function, change that part and add a test in your tests/ or frontend test file. Do not rename shared fields, duplicate common code or rewrite other modules. Ask the Scrum Master before changing shared contracts, storage or provider code.

What should I see?

Expect basic/background fields, import preview and replacement choice, a source-linked skill passport, twelve Unknown/1-10 controls and Save profile. Show persistence after reload; a success toast alone is insufficient.

Load synthetic inputs supplies pre-reviewed Mira records for independent operation. It is not fresh extraction. The tiny Asha terminal examples and the richer Mira browser examples are intentionally different inputs.

6. How do I know it works?

```
.\profile_builder\run.ps1 example
.\profile_builder\run.ps1 test
```

example runs the small teaching case in a fresh temporary database and checks its result. test runs your real standalone API check, Python tests, React tests, type checking, lint, your production build and your browser test. It stops on failure. Both use synthetic data; tests force rules mode and do not prove live LLM quality.

- Manual save: Enter samples/input.json values. Save/reload: name, education and preferred role remain. Compare output.json; IDs vary, twelve null ratings do not. Attach samples/cv-upload-SYNTHETIC.pdf and check the saved CV opens correctly.
- Import: Read samples/reviewed-resume.json, import-request.json and import-preview.json. Load synthetic inputs in the browser and import its reviewed resume. Preview must not silently save or rate skills.
- Manual conflict: Enter your own education, import without Replace and check it is preserved. Select Replace intentionally and check the preview changes before saving.
- Invalid / old save: Try a blank name, invalid email and ratings outside 1-10. Expect validation. A stale revision returns 409; keep the draft, reload and reconcile the changes.
- Assessments / GPS: Save a candidate and use the same ID in both assessments. Run a what-if and return: no hypothetical skills or ratings may appear in the saved profile.

Use an unfamiliar document too

Use only documents you own or have permission to process. Before running extraction, list expected skills and supporting words yourself. Compare missing and invented skills, sections, groups and quotes. A successful API response or valid JSON is not proof of correct extraction. Do not commit real CVs or send them to Groq without explicit permission.

If something fails

Read the FIRST error. Missing Python/npm: install and reopen the terminal. Missing packages: rerun setup. Browser cannot reach API: keep api running. Port busy: stop your own earlier server or coordinate a free port; do not kill unknown processes. Wrong extraction: compare the source, review and add a regression case.

7. Missing resources and what to do

What you can do now

All local teaching inputs are synthetic. No organizer pack, API key or cloud account is needed to run, edit and test your module. The tiny samples teach fields and behavior; the larger reconstructed pack remains under data/synthetic/. Neither is official organizer data.

Your samples - every file has a purpose

- cv-upload-SYNTHETIC.pdf - Fictional Mira CV for checking Profile Builder's direct CV attachment.
- import-preview.json - Actual import result; it remains a draft until explicitly saved.
- import-request.json - Full import-preview request with resume ID and profile draft.
- input.json - Complete upload description or JSON request shown in this guide.
- output.json - Complete response captured from that actual API request.
- reviewed-resume.json - Reviewed Mira input snapshot for understanding import.

Other modules and cloud resources

The shared folder and locked root dependencies are required; clone the whole repository. The local workspace supplies reviewed upstream snapshots, so you do not wait for a teammate's server. Later, integration uses real saved IDs, versions and owned source links, not the snapshot UUIDs in this guide.

Supabase accounts, live database rules and real organizer data are configured and verified by the Scrum Master when authorized. Local SQLite storage already works. Module 3 needs no LLM; Module 5's named-JD matching also needs no LLM or company data. Company-readiness previews remain unavailable without a valid chosen benchmark.

8. Submit and explain your module

Completion checklist

[ ] I can run/explain it. [ ] Required inputs/outputs work. [ ] Normal/failure cases checked. [ ] Tests passed or failures reported. [ ] Contracts/source links preserved. [ ] No secrets/private CVs committed.

Push to YOUR own GitHub

Create an EMPTY private repository on your own account. From your full working clone, replace the URL below with YOUR new repository, review/stage only your changes, then run:

```
git remote rename origin scrum-master
git remote add origin https://github.com/YOU/YOUR-REPO.git
git add profile_builder
.\.venv\Scripts\python.exe scripts/scan_handoff.py --staged
git commit -m "Verify my RADIX module"
git push -u origin main
```

Keep the whole working clone, including your source/tests/samples, guides, launcher and shared support. Never force-push. Grant the Scrum Master access if private.

Return URL, git rev-parse HEAD, changed files, actual test commands/results, three normal + three failure cases, synthetic screenshots and limitations. Distinguish implemented, mock-tested, live-verified and pending. The Scrum Master handles integration; you need not solve central merges.

A 45-second explanation for judges

I own the candidate's saved profile. A person can type details or start from a reviewed resume, then add projects, hackathons, internships, certificates and preferred roles. Import first creates a draft, so it does not silently overwrite saved work. The profile keeps evidence links and separate optional self-ratings; Unknown stays unknown. Both assessments load the same saved revision. I verify save/reload, import conflicts and invalid entries. The current passport is persistent, but it is not an immutable history timeline.

Three likely questions

- Does import save automatically? No. The user inspects the merged draft and explicitly saves it.
- Why are ratings Unknown? A CV mention does not measure proficiency. Only an explicit self-rating sets a number.
- What happens if two edits conflict? An old revision is rejected with 409 so the user can reload and reconcile.

Copy this prompt into your AI assistant

Help me complete module 3: Profile Builder step by step using this guide. First run it; request the exact section-5 source files and tests before editing. Preserve working code and shared fields. Fix a reproduced issue and test it. Keep confidence separate from competency and scenarios separate from saved profiles. Never invent official data or request pasted keys. Help me submit my own GitHub URL, tested commit, results and honest limitations.